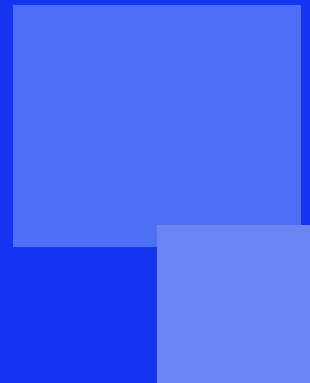


{EPITECH}



HOMEMIND ROBOT_

< BUILD YOUR ROBOT FROM SCRATCH />

HOMEMIND ROBOT

deliverable name:	robot_ws/ (workspace ROS 2)
language:	Python 3.11+ / C++ (optionnel — nodes temps-réel)
compilation:	colcon build
Authorized tools:	ROS 2 · PyTorch · Gazebo · N8N (bonus)

Présentation

HomeMind Robot est un projet de robotique personnelle de A à Z. L'objectif est de concevoir, imprimer en 3D, câbler et programmer un robot autonome capable d'évoluer dans un environnement domestique inconnu, puis d'y embarquer progressivement une intelligence artificielle apprenante.

Ce projet n'existe pas encore — c'est toi qui vas le construire.

La rupture mathématique

Passer d'une IA à règles hardcodées à une IA apprenante, c'est une vraie rupture mathématique — pas juste une question de hardware (meilleur proc, caméra).

Niveau	Ce que tu codes	Maths requis
Heuristique	Règles if/else, FSM, A*	Logique, graphes
Reinforcement Learning	Agent optimisant une reward	Dérivées, tenseurs, proba
Vision — CNN	Détection d'objets temps réel	Convolutions, géom. proj.
LLM embarqué	Raisonnement langage naturel	Attention, embeddings



La caméra et le meilleur processeur sont des ressources. Le vrai saut c'est le modèle mathématique derrière — pas le matériel.

Stack technologique

Composant	Technologie
Langage principal	Python 3.11+
Middleware robot	ROS 2 Humble
Simulation	Gazebo Fortress / PyBullet
IA / Machine Learning	PyTorch + stable-baselines3 + Gymnasium
Vision	OpenCV + YOLOv8 nano
SLAM / Navigation	slam_toolbox + nav2
LLM embarqué (bonus)	Ollama + Mistral 7B quantisé

Hardware	Raspberry Pi 5 · Imprimante 3D · L298N · HC-SR04
----------	--

Phases du projet

0

Bootstrap hardware

PHASE

~3 semaines

Concevoir et imprimer le châssis, câbler les moteurs, faire tourner ton premier script Python qui bouge une roue. Zéro IA ici — juste électronique et CAD.

- Modéliser le châssis dans FreeCAD ou Fusion 360, imprimer les pièces
- Câbler les moteurs DC / servo avec un driver L298N
- Installer ROS 2 sur Raspberry Pi, créer un node publiant sur `/cmd_vel`
- Ajouter des capteurs ultrason HC-SR04 pour détecter les obstacles

Livrable : Le robot roule en télécommande depuis un terminal Python.

Stack : FreeCAD · RPi.GPIO · ROS 2 · rclpy

1

IA heuristique — règles hardcodées

PHASE

~2 semaines

Tu codes la logique du robot à la main via un automate fini (FSM). Pas de l'IA au sens strict — de l'ingénierie de règles. La base indispensable pour comprendre ce que l'apprentissage remplacera.

- Implémenter un automate fini (FSM) : états EXPLORE / ÉVITE / STOP
- Coder la navigation par wall-following avec les capteurs ultrason
- Écrire un pathfinding A* sur une grille simple (ton salon = la carte)
- Logger les décisions dans un fichier CSV pour analyser les comportements

Maths : Logique booléenne, graphes, $A^ = f(n) = g(n) + h(n)$. Algorithmique pure.*

Livrable : Le robot évite les murs et atteint un point B depuis un point A.

Stack : Python FSM · heapq (A*) · ROS 2 nav2

2

IA apprenante — reinforcement learning

PHASE

~6 semaines

Le maths change de nature ici. Tu remplaces tes règles if/else par un agent qui optimise une fonction de récompense. Il apprend par essai-erreur — tu ne lui dis plus quoi faire, tu lui dis ce qui est bien ou mal.

- Créer un environnement Gym custom simulant ton salon (Gazebo ou PyBullet)
- Implémenter PPO (Proximal Policy Optimization) avec stable-baselines3
- Définir la reward function : +1 avance, -10 collision, +100 destination
- Transférer le modèle entraîné en sim vers le vrai robot (sim-to-real)
- Ajouter une cam + MobileNet pour identifier les objets dans la pièce

Maths : Algèbre linéaire (tenseurs), calcul différentiel (backprop = dL/dw), probabilités (politique stochastique), optimisation par gradient.

Livable : Le robot apprend à naviguer dans une pièce inconnue sans règles hardcodées.

Stack : PyTorch · stable-baselines3 · Gymnasium · Gazebo · OpenCV

3

Perception — vision et mapping

PHASE

~4 semaines

Le robot construit une carte de son environnement en temps réel et reconnaît les objets. La brique qui rend le comportement visuellement intelligent.

- Intégrer une caméra depth (OAK-D Lite ou Intel RealSense D435) sur le châssis
- Implémenter SLAM avec slam_toolbox (Simultaneous Localization and Mapping)
- Utiliser YOLOv8 nano (tourne sur RPi) pour la détection en temps réel
- Fusionner les données cam + ultrason dans un costmap ROS 2

*Maths : Géométrie projective (matrices intrinsèques cam), filtres de Kalman, CNN = convolutions = $ReLU(W * x + b)$.*

Livable : Le robot crée une carte 2D de ta maison et identifie les meubles.

Stack : YOLOv8 · slam_toolbox · OAK-D Lite · nav2

4

Couche langage — proto-conscience

PHASE

bonus / recherche

Tu embarques un LLM local pour que le robot raisonne en langage naturel sur ses perceptions. Pas la conscience — mais la frontière actuelle des labos.

- Faire tourner Ollama sur un mini-PC attaché au robot (ou offload WiFi)
- Construire un pipeline perception → prompt → action
- Implémenter une mémoire épisodique simple (SQLite)
- Lier les actions LLM aux commandes ROS 2 via LangChain ou script custom

*Maths : Attention transformers ($Attention(Q,K,V) = softmax(QK^T/sqrt(d))*V$), embeddings vectoriels, similarité cosinus.*

Livable : Tu dis 'va dans la cuisine et attends' — il comprend, navigue, attend.

Stack : Ollama · LangChain · SQLite · Mistral 7B

Features additionnelles

Voici des features à implémenter une fois les phases de base fonctionnelles. Sélectionne celles qui t'intéressent — chacune a une valeur pédagogique propre.

Le robot pourrait

- Cartographier un appartement entier en une session autonome (SLAM multi-pièce)
- Reconnaître les personnes dans la pièce via FaceNet embarqué
- Communiquer via voix (Whisper pour la reconnaissance, TTS pour la réponse)
- Recharger sa batterie de façon autonome en trouvant sa base de charge
- Envoyer des alertes (Telegram, email) si il détecte un événement anormal

Tu pourrais

- Réaliser un audit de sécurité du réseau WiFi du robot
- Publier ton modèle RL sur HuggingFace Hub
- Comparer plusieurs architectures de reward function et documenter les résultats
- Optimiser le pipeline de vision pour minimiser la latence sur RPi



N'oublie pas que certains modèles IA offrent des crédits gratuits pour étudiants. La qualité du modèle n'est pas un critère d'évaluation — la qualité de l'intégration l'est.

Matériel minimum — Phase 0+1

Composant	Modèle recommandé	Prix
Imprimante 3D	Bambu Lab A1 Mini ou Ender 3 V3	200–500 €
Ordinateur embarqué	Raspberry Pi 5 — 4 GB minimum	~80 €
Driver moteur	L298N	~5 €
Moteurs DC	Moteurs DC 6V avec encodeurs	~20 €
Capteurs ultrason	HC-SR04 x3	~9 €
Batterie	LiPo 7.4V 2S 2200mAh	~25 €
Filament 3D	PLA 1kg	~25 €
Total		~365–665 €